PROMETEO

Proyecto Intermodular 1º DAW

Participantes

Jheremi Loachamin Lema

Juan Camilo Montero

Índice

1. **Introducción**
   1.1Objetivos del proyecto
   1.2 Justificación
2. **Descripción del proyecto**
   2.1 Idea general del sistema
   2.2 Funcionalidades principales
   2.3 Alcance del proyecto
3. **Tecnologías utilizadas**
   3.1 Lenguajes de programación
   3.2 Herramientas de desarrollo
   3.3 Librerías o Frameworks utilizados
4. **Análisis del sistema**
   4.1 Identificación de necesidades
   4.2 Requisitos funcionales
   4.3 Requisitos no funcionales
5. **Diseño del sistema**
   5.1 Arquitectura del proyecto
   5.2 Diseño de la base de datos
   5.3 Estructura del proyecto
6. **Desarrollo del proyecto**
   6.2 Implementación de la interfaz web
   6.3 Integración entre frontend y backend
7. **Pruebas y validación**
   7.1 Pruebas realizadas
   7.2 Resultados obtenidos
8. **Conclusiones**
   8.1 Resultados del proyecto
   8.2 Mejoras futuras
9. **Bibliografía y recursos**
10. **Anexos**

1. Introducción

En la actualidad, la digitalización de los negocios se ha convertido en un elemento fundamental para mejorar la organización y la gestión de los servicios que ofrecen, muchos establecimientos deciden invertir sus recursos en sistemas informáticos para gestionar los servicios de una manera más correcta, por ejemplo, las barberías. Utilizan sistemas informáticos para gestionar reservas, clientes y servicios de forma más eficiente.

El presente proyecto, denominado BarberAPI, consiste en el desarrollo de una aplicación web que permite gestionar los principales elementos de una Barberia. A través de este sistema es posible registrar clientes, administrar citas y organizar los servicios ofrecidos por la Barberia.

Este proyecto se ha desarrollado con el objetivo de aplicar los conocimientos adquiridos en los distintos módulos del ciclo formativo, integrando conceptos de programación, lenguaje de marcas, gestión de datos, diferentes entornos o sistemas donde se pueda visualizar la aplicación y la presentación personal del proyecto y los contribuyentes.

1.1 Objetivos del proyecto

El objetivo de este proyecto es desarrollar una web para la gestión de una Barberia, permitiendo administrar información relacionada con **clientes**, **citas** y **servicios**.

Entre los *objetivos específicos* se encuentran:

- *Diseñar e implementar una web funcional para gestionar los servicios de una Barberia*.
- *Permitir la gestión de clientes y reservas de citas*.
- *Aplicar conocimientos de programación, desarrollo web, bases de datos, sistemas informáticos y documentación*.
- *Organizar el proyecto siguiente una estructura clara y modular*.
- *Utilizar herramientas de desarrollo y control versiones como Git y GitHub*.

- *Simular el funcionamiento de un sistema real de gestión de barberías*.

1.2 Justificación

La realización de este proyecto permite integrar y poner en práctica los conocimientos adquiridos durante el ciclo formativo. El desarrollo de BarberAPI tiene como finalidad crear una web que simule un sistema real utilizado con muchos negocios para gestionar sus servicios y clientes.

Además, este proyecto permite trabajar diferentes aspectos del desarrollo de software, como el diseño web, la organización de código, la documentación del proyecto y la utilización de herramientas colaborativas como GitHub. Cabe recalcar que BarberAPI será de código abierto para toda la comunidad que desee colaborar con un mejor desarrollo y proyección.

De esta forma, el proyecto no solo cumple un objetivo académico, sino que también representa una experiencia práctica que ayuda a comprender como se desarrollan aplicaciones reales en el ámbito profesional.

2. Descripción del Proyecto

2.1 Idea General del Sistema

El sistema "BarberAPI" nace como una plataforma digital integra que tiene como objetivo optimizar y modernizar la gestión diaria de una barbería. La idea principal es garantizar un entorno unificado donde se encuentren las necesidades de los clientes y la administración interna del negocio.

El proyecto se divide en dos grandes bloques:

- **Portal WEB Orientado al Cliente**: Una interfaz pública, accesible e intuitiva, cuya funcionalidad es ser una herramienta digital de la barbería. A través del portal, los usuarios podrán informarse sobre los servicios que se ofrecen en BarberAPI, conocer tarifas, visualizar la información de contacto y, principalmente, realizar reservas de citas de forma autónoma, sin depender de llamadas telefónicas o tiempos de espera.
- **Sistema de Gestión Interna (BackEnd)**: El núcleo lógico de la aplicación, encargado de procesar la información de manera segura y eficiente. Este sistema interactuará de forma directa con una base de

datos centralizada, permitiendo a los administradores del negocio (los barberos o gerentes) tener un control total sobre:

- **Gestión de Citas**: Visualizar, confirmar, modificar o cancelar las reservas realizadas por los clientes.

Por consiguiente, la idea general es crear una herramienta que automatice las tareas rutinarias, mejore la experiencia con el usuario, actualice aun sistema informático que organice las operaciones diarias de la barbería.

2.2 Funcionalidades Principales

El sistema de BarberAPI está diseñado para poder ofrecer un conjunto de herramientas que cubran tanto las necesidades de los usuarios finales como la del personal de la barbería, cuyas funcionalidades del sistema se dividen en dos módulos principales según el rol del usuario.

A. Modulo del Cliente (Interfaz Pública)

- **Catálogo de servicios:** Visualización de los distintos servicios que ofrece la barbería (corte de pelo, arreglo de barba, tintes, etc.), incluyendo su descripción, duración estimada y precio.
- **Gestión de reservas (CRUD):** Permite al cliente seleccionar un servicio, elegir al profesional (barbero) de su preferencia, y escoger una fecha y hora disponibles para agendar su cita, en caso de no poder cambiar horario, cancelar cita o buscar su cita.
- **Información corporativa:** Acceso a datos de contacto, ubicación del local, horarios de apertura y enlaces a redes sociales.

2.3 Alcance del proyecto

El proyecto BarberAPI abarca el diseño, desarrollo y documentación de un sistema funcional que simula la gestión de una barbería real. El desarrollo se limitará a las competencias adquiridas durante el primer año del ciclo de Desarrollo de Aplicaciones Web (DAW).

Alcance dentro del proyecto:

- **Diseño e implementación de la Base de Datos:** Creación de un modelo relacional en SQL que almacene de forma estructurada la información de clientes, empleados (barberos), catálogo de servicios y registro de reservas.

- **Desarrollo del Frontend:** Creación de una interfaz web corporativa estática utilizando HTML semántico y CSS, asegurando una navegación lógica y un diseño *responsive* básico (adaptación a dispositivos móviles).
- **Desarrollo del Backend:** Programación de la lógica de negocio mediante una aplicación en Java, aplicando los principios de la Programación Orientada a Objetos (POO).
- **Conectividad a Datos:** Integración del backend con la base de datos a través de JDBC para permitir operaciones CRUD (Crear, Leer, Actualizar, Borrar) sobre los registros del sistema.
- **Documentación Técnica y Despliegue:** Elaboración de un informe técnico sobre los requisitos del sistema y el entorno de ejecución necesario para hacer funcionar la aplicación de manera local.

Fuera del alcance del proyecto (Limitaciones):

Debido a que el sistema tiene un enfoque puramente académico y demostrativo para el primer año del ciclo formativo.

3. Tecnologías utilizadas

Para el desarrollo del sistema BarberAPI, se ha seleccionado un conjunto de tecnologías empleadas como requerimientos técnicos del primer curso del ciclo de Desarrollo de Aplicaciones Web (DAW).

3.1 Lenguajes de programación

- **Java:** Lenguaje principal utilizado para el desarrollo del *Backend* y la lógica de negocio de la aplicación. Se ha aplicado el paradigma de Programación Orientada a Objetos (POO) para estructurar el código de manera eficiente, segura y escalable.
- **SQL (Structured Query Language):** Diseño, creación y manipulación de la base de datos relacional del sistema. Mediante query ´s SQL que definen las tablas de clientes, servicios y reservas, así como la inserción de datos de prueba.
- **HTML5 y CSS3:** Lenguajes de marcas y estilos empleados para la maquetación y el diseño visual de la interfaz web del cliente. Garantizan una estructura semántica correcta y un diseño coherente y adaptable.

3.2 Herramientas de desarrollo

- **Git y GitHub:** Sistema de control de versiones distribuido y plataforma de alojamiento de repositorios, respectivamente. Utilizados para la gestión del código fuente, el trabajo colaborativo en ramas (*branches*), la integración de cambios (*Pull Requests*) y el mantenimiento de un historial de *commits* coherente.
- **Entorno de Desarrollo Integrado (IDE):** *IntelliJ IDEA y Visual Studio Code*. Utilizado para la escritura, depuración y compilación del código Java y el desarrollo web.
- **Sistema Gestor de Bases de Datos (SGBD):** *MySQL Workbench*. Empleado para administrar y visualizar el modelo relacional del proyecto.

3.3 Librerías o frameworks utilizados

La principal tecnología de integración utilizada es:

- **JDBC (Java Database Connectivity):** API de Java requerida de forma obligatoria para establecer la conexión entre la aplicación *Backend* y la base de datos relacional. Permite ejecutar las sentencias SQL (operaciones CRUD) directamente desde el código Java.
- **Springboot:** Es un framework de Java, el cual facilita y simplifica la creación, configuración y despliegue de aplicaciones web y microservicios independientes.

4. Análisis del sistema

Este apartado detalla el proceso de estudio previo a la programación, donde se han identificado las posibles debilidades encontradas en los modelos de gestión tradicional de una barbería y se han establecido las soluciones para que el sistema BarberAPI pueda lograr sus objetivos.

4.1 Identificación de necesidades

El diseño de BarberAPI surge de la observación de los problemas comunes a los que se enfrenta una barbería que gestiona sus citas de forma manual o analógica (por ejemplo, usando una agenda de papel o recibiendo reservas exclusivamente por teléfono o mensajes):

- **Dependencia del horario comercial:** Los clientes solo pueden reservar cuando el local está abierto y alguien puede atender el teléfono, lo que limita la captación de reservas.

- **Sobrecarga de trabajo:** El personal debe interrumpir constantemente sus servicios para atender llamadas y apuntar citas.
- **Errores humanos:** Al anotar a mano, es frecuente que haya solapamientos de horarios, pérdida de datos de contacto o confusiones sobre el servicio solicitado.
- **Falta de autonomía del cliente:** El usuario no tiene una forma rápida de consultar sus citas programadas o cancelarlas sin tener que contactar directamente con el local.

BarberAPI nace para digitalizar este proceso, centralizando la información en una base de datos y dándole al cliente la capacidad de autogestionar sus reservas en cualquier momento.

4.2 Requisitos funcionales

Los requisitos funcionales describen las acciones específicas que el sistema (la aplicación Java conectada a la base de datos) debe ser capaz de realizar. Basado en el alcance definido, el sistema debe cumplir con los siguientes puntos:

- **RF-01 (Gestión de Servicios):** El sistema debe permitir consultar el catálogo de servicios disponibles (corte, barba, tintes, etc.) almacenados en la base de datos, mostrando su precio y duración.
- **RF-02 (Registro de Clientes):** El sistema debe permitir almacenar y consultar los datos de los clientes (nombre, teléfono, correo) para asociarlos a sus respectivas reservas.
- **RF-03 (Creación de Reservas):** El sistema debe permitir a un cliente registrar una nueva cita, seleccionando el servicio deseado, la fecha y la hora (Operación *Create* del CRUD).
- **RF-04 (Consulta de Reservas):** El sistema debe permitir a un cliente buscar y visualizar los detalles de su cita programada (Operación *Read* del CRUD).
- **RF-05 (Modificación de Reservas):** El sistema debe permitir actualizar una reserva existente, cambiando el horario o el servicio seleccionado (Operación *Update* del CRUD).
- **RF-06 (Cancelación de Reservas):** El sistema debe permitir eliminar una reserva programada si el cliente decide cancelarla (Operación *Delete* del CRUD).

4.3 Requisitos no funcionales

Los requisitos no funcionales definen los atributos de calidad, rendimiento y restricciones del entorno del sistema:

- **RNF-01 (Usabilidad Web):** La interfaz web corporativa debe ser intuitiva, con una navegación clara y un diseño *responsive* básico que permita su correcta visualización tanto en monitores de escritorio como en dispositivos móviles.
- **RNF-02 (Persistencia de Datos):** Toda la información debe almacenarse de forma persistente y estructurada en una base de datos relacional MySQL.
- **RNF-03 (Arquitectura de Software):** El código backend (Java) debe seguir los principios de la Programación Orientada a Objetos, garantizando la modularidad y separación de responsabilidades (por ejemplo, separando la lógica de conexión a la base de datos del resto de la aplicación).
- **RNF-04 (Entorno de Ejecución):** El sistema debe estar preparado para ejecutarse y probarse en un entorno local, requiriendo un SGBD activo y la configuración correcta del Driver JDBC.

4.4. Configuración y resolución de incidencias del entorno"

- **Descripción del problema:** Error de compilación en el entorno de maven. Incompatibilidad con la ejecución “mvn”: `invalid target release: 17`.
- **Causa técnica:** El sistema operativo tenía configurada la variable `JAVA_HOME` apuntando a Java 8 (versión 52.0), mientras que el proyecto requería Java 17 o superior (versión 61.0 o 63.0) para funcionar con Spring Boot.
- **Solución aplicada:** Se instala la dependencia que apunta a la versión 17 o superior en el SDK de IntelliJ con la versión del compilador en el archivo pom.xml. Desde dispositivos Windows se debe gestionar las variables de entorno apuntando al directorio donde está la carpeta con el SDK 17 o superior.

4.5 Requisitos del sistema y entorno de ejecución

Para garantizar el correcto funcionamiento y despliegue del sistema **BarberAPI**, se han establecido los requisitos mínimos y recomendados tanto para el entorno del servidor

(donde se aloja y ejecuta la lógica de negocio y la base de datos) como para el entorno del cliente (dispositivo del usuario final).

A. Requisitos del Servidor (Entorno de Desarrollo y Ejecución Local) Al tratarse de una aplicación desarrollada en Java y respaldada por una base de datos relacionales, el equipo que compile y aloje el proyecto debe contar con las siguientes características:

- **Hardware mínimo:**
  - Procesador: Dual-Core a 2.0 GHz o superior.
  - Memoria RAM: 4 GB (Se recomiendan 8 GB para mantener una ejecución fluida del IDE y del servidor de base de datos de manera simultánea).
  - Almacenamiento: 2 GB de espacio libre en disco.
- **Software necesario:**
  - Sistema Operativo: Windows 10/11, macOS o distribuciones Linux recientes.
  - Java Development Kit (JDK): Versión 17 o superior (Requisito estricto, debidamente configurado en las variables de entorno JAVA_HOME).
  - Sistema Gestor de Base de Datos: MySQL Server 8.0 o superior.
  - Driver de conexión: MySQL Connector/J compatible con la versión de la base de datos.

B. Requisitos del Cliente (Usuario Final) Dado que la capa de presentación (Frontend) es una interfaz web estática renderizada en el navegador, los requisitos para el cliente son altamente accesibles, garantizando la máxima inclusión de usuarios:

- **Hardware:**
  - Cualquier dispositivo con capacidad de navegación web: Ordenador de sobremesa, ordenador portátil, tableta o *smartphone*.
- **Software y Pantalla:**
  - Navegadores web compatibles: Google Chrome, Mozilla Firefox, Safari o Microsoft Edge en sus versiones actualizadas (con soporte pleno para HTML5 y CSS3).
  - Resolución de pantalla: Sistema *responsive* diseñado para adaptarse a pantallas desde resoluciones móviles (mínimo de 320px de ancho) hasta monitores *ultrawide*.

- Conexión: Conexión a la red local (localhost) para la fase de pruebas, o conexión a Internet estándar si el proyecto es desplegado en un servidor de producción.

5. Diseño del sistema

En esta fase se definen los cimientos técnicos de BarberAPI, estableciendo la arquitectura de software elegida, el modelo de datos que soportará la información y la organización lógica de los archivos del proyecto.

5.1 Arquitectura del proyecto

El sistema BarberAPI se ha diseñado siguiendo una arquitectura monolítica basada en capas, lo que facilita la separación de responsabilidades y el mantenimiento del código, adaptándose a los estándares exigidos en el primer curso de DAW. Las capas principales son:

- **Capa de Presentación (Frontend):** Compuesta por los archivos estáticos HTML5 y CSS3. Es la interfaz gráfica mediante la cual el cliente interactúa con el catálogo de servicios y la información de la barbería.
- **Capa de Lógica de Negocio (Backend):** Desarrollada en Java. Contiene las clases y métodos que procesan las reglas del negocio, como la validación de datos al crear una reserva o la gestión de los menús de la aplicación.
- **Capa de Acceso a Datos:** Actúa como puente entre la lógica de negocio y la base de datos. Utiliza la API JDBC de Java para enviar sentencias SQL y recuperar los resultados.
- **Capa de Persistencia (Base de Datos):** El servidor MySQL donde reside la información estructurada de la aplicación, garantizando que los datos de citas y clientes no se pierdan al cerrar el programa.

5.2 Diseño de la base de datos

Para el almacenamiento de la información, se ha optado por una base de datos relacional. El diseño se compone del correspondiente Diagrama Entidad-Relación (E/R) y su paso al modelo relacional.

Entidades principales y sus atributos:

1. **Tabla clientes**: Almacena la información de los usuarios que realizan las reservas.
    a. id_cliente (Primary Key, Autoincremental): Identificador único del cliente.
    b. nombre: Nombre del cliente.
    c. apellidos: Apellidos del cliente.
    d. telefono: Teléfono de contacto.
    e. email: Correo electrónico.
2. **Tabla servicios**: Contiene el catálogo de tratamientos que ofrece la barbería.
    a. id_servicio (Primary Key, Autoincremental): Identificador único del servicio.
    b. nombre_servicio: Nombre (ej. Corte clásico, Arreglo de barba).
    c. descripcion: Breve detalle de lo que incluye.
    d. precio: Coste del servicio (Decimal).
    e. duracion_minutos: Tiempo estimado que tarda el barbero en realizarlo.
3. **Tabla reservas**: Es la tabla transaccional que une a los clientes con los servicios elegidos en una fecha concreta.
    a. id_reserva (Primary Key, Autoincremental): Identificador único de la cita.
    b. id_cliente (Foreign Key): Relaciona la cita con el cliente que la solicita.
    c. id_servicio (Foreign Key): Relaciona la cita con el tratamiento elegido.
    d. fecha_hora: Fecha y hora exacta de la cita (Datatype: DATETIME).
    e. estado: Situación de la reserva (ej. 'Pendiente', 'Completada', 'Cancelada').

5.3 Estructura del proyecto

Para garantizar un entorno de desarrollo organizado y colaborativo mediante Git/GitHub, el repositorio del proyecto se ha estructurado en los siguientes directorios principales:

- `/web`: Contiene todos los archivos de la interfaz de usuario (Lenguajes de Marcas). Incluye los ficheros `.html`, las hojas de estilo `.css` y la carpeta `/assets` para las imágenes del negocio.
- `/src`: Directorio raíz del código fuente en Java. En su interior, el código se divide en paquetes lógicos.

- `/sql`: Almacena los scripts de base de datos (`creacion_tablas.sql`, `insercion_datos.sql`) necesarios para recrear el entorno de persistencia en cualquier equipo.
- `/docs`: Destinado a la documentación del proyecto, incluyendo este informe técnico, diagramas, manuales de usuario y los documentos del perfil profesional requeridos para el módulo de Empleabilidad.

6. Desarrollo del proyecto

Este apartado describe el proceso de construcción técnica de BarberAPI, detallando cómo se ha programado la interfaz visual, la lógica de negocio y cómo interactúan los diferentes componentes del sistema para cumplir con los requisitos establecidos.

6.1 Implementación de la interfaz web (Frontend)

La interfaz pública del cliente se ha desarrollado buscando un equilibrio entre la estética visual, la usabilidad y el cumplimiento estricto de los estándares web.

- **Estructura con HTML5:** Se han creado las diferentes vistas de la aplicación utilizando etiquetas semánticas (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`) para garantizar que el código sea accesible y fácil de leer. La web incluye la página de inicio, el catálogo de servicios (tablas o tarjetas) y los formularios de registro y reserva.
- **Diseño visual con CSS3:** Se ha definido una hoja de estilos centralizada que aporta la identidad corporativa a BarberAPI. Se han utilizado variables CSS para los colores principales (tonos propios de una barbería clásica o moderna) y se ha aplicado un modelo de cajas (*Flexbox* o *Grid*) para alinear los elementos.
- **Funcionalidad JavaScript:** Para realizar la web de forma más interactiva y que sea más inmersiva con el usuario/cliente, se agregó componentes JavaScript en el apartado del calendario de la web y la reserva de las horas. Esta experiencia hace que el usuario pueda reservar datos reales y con horas disponibles. En caso de no estar disponibles estas se bloquean. También se capturan los datos del formulario para enviárselos a la DB y se gestionan por JAVA.
- **Adaptabilidad (Responsive Design):** Mediante el uso de *Media Queries* en CSS, se ha garantizado que la web ofrezca una experiencia de navegación correcta tanto en resoluciones de escritorio como en

dispositivos móviles, reestructurando los menús y apilando las columnas cuando es necesario.

6.2 Implementación de la Lógica Backend

El "Backend" o motor de la aplicación se ha construido utilizando Java Core, aplicando los principios de la Programación Orientada a Objetos (POO). El desarrollo se ha estructurado en las siguientes fases:

- **Creación del Modelo de Datos (Clases):** Se han programado las clases Java correspondientes a las entidades de la base de datos (`Cliente`, `Servicio`, `Reserva`). Estas clases utilizan encapsulamiento (atributos privados y métodos *getters/setters*) para proteger la información en memoria.
- **Desarrollo del DAO (Data Access Object):** Se ha implementado un patrón de diseño básico para separar la lógica de conexión a la base de datos del resto del programa. En estas clases se concentran las operaciones CRUD.
- **Conexión mediante JDBC:** Se ha utilizado el *driver* JDBC de MySQL para establecer el puente entre la aplicación Java y la base de datos. A través de la clase Connection, el sistema envía las sentencias SQL (previamente diseñadas con INSERT, UPDATE, DELETE y SELECT) y procesa los ResultSet devueltos para transformarlos en objetos Java utilizables por el programa.
- **Framework "Springboot":** Se empleo este Framework, para combinar la lógica del backend de JAVA con el frontend de Html-CSS. Springboot es una tecnologia moderna que permite conectar estos ámbitos creando webs modernas, favoreciendo la implementación del lenguaje de marcas de HTML y estilos CSS con la lógica de JAVA.

6.3 Integración entre Frontend y Backend

Para el nivel del primer curso de DAW, la integración de los sistemas se enfoca en la coherencia de los datos y el funcionamiento unificado del proyecto global.

- **Flujo de la información:** La página web actúa como el punto de entrada de la información. El catálogo de servicios mostrado en el HTML refleja

exactamente los datos parametrizados en la tabla `servicios` de la base de datos.

- **Simulación de la operativa transaccional:** El usuario interactúa con los formularios web para solicitar una cita. Posteriormente, el motor backend en Java es el encargado de recoger esos datos (ya sea mediante integración directa si se utilizan Servlets/JSP, o mediante simulación en consola/interfaz de escritorio en Java) y validarlos antes de inyectarlos en la base de datos MySQL mediante sentencias preparadas (`PreparedStatement` en JDBC), evitando así vulnerabilidades como la inyección SQL.
- **Control de errores:** El Backend está programado para capturar excepciones (como fallos de conexión a la BBDD o intentos de reservar en horas ocupadas) y gestionar el flujo del programa para que no se interrumpa abruptamente.

7. Pruebas y validación.

7.1 Pruebas realizadas

- **1.1-** *"Se detectó una excepción SQL`Exception` al encadenar múltiples operaciones CRUD debido al cierre prematuro de la conexión estática. Se resolvió implementando una validación `isClosed()` en el patrón Singleton para garantizar la disponibilidad del recurso en todo el ciclo de vida de la aplicación".* (Anexo 2)
- **1.2 -** "Se realizó una prueba de reserva a través de un dispositivo móvil, donde se rellenó el formulario con los datos correspondientes y la fecha y la hora". (Anexo 4)
- **1.3 -** *"Prueba de disposición de maquetación styles.css y html. Cuando la web se visualizaba en dispositivos más pequeños los margin y padding ocupaban demasiado espacio y se producía un desbordamiento" (Anexo 5)*

7.2 resultados Obtenidos

- **1.1 -** Par solucionarlo, dentro de del entorno de JAVA se estaba ejecutando con JAVA 8 no con JAVA 17 o superior, por lo que cuando se quería compilar esto producía ese error de compilación, ya que las nuevas funcionalidades

asignadas a JAVA 17 no se aplicaban al compilador. Se procedió a cambiar las variables de entorno donde se ejecutará en el "PATH/" la nueva versión. Solucionando así el problema (Anexo 3)

- **1.2 -** Para llevar a cabo la solución, se corrigieron pequeños errores de código en ServiciosDAO, ClienteDAO, ReservasDAO, ClienteController. Como el proyecto se ejecuta en producción real se modificaron las credenciales de acceso a la base de datos, esto permitía la conexión exitosa a la base de datos. Finalmente realizando estos cambios permitió la creación de una "reserva exitosa".
- **1.3 -** Solución llevada a cabo, se modificaron las **medias queries**, validando y reduciendo la disposición de los elementos con tamaños un poco más reducidos. Esto hace que no ocupen tanto espacio en el elemento padre {html}, pudiendo adaptarse a dichos dispositivos. *(Anexo 6)*.

8.Conclusiones

8.1 Resultados del proyecto

El desarrollo del sistema BarberAPI ha concluido de manera satisfactoria, logrando cumplir con todos los objetivos principales, tanto técnicos como académicos, establecidos al inicio del documento. Se ha construido una aplicación web funcional, robusta y escalable que simula con precisión la operativa de reservas de una barbería moderna.

Entre los principales logros del proyecto destacan:

- **Integración Full-Stack:** Se ha logrado una comunicación bidireccional eficiente entre la capa de presentación (Frontend con HTML5, CSS3 y JavaScript) y la lógica de negocio (Backend en Java con Spring Boot), garantizando un flujo de datos seguro hacia la base de datos MySQL mediante JDBC. (Anexo 7)
- **Experiencia de Usuario (UX) y Usabilidad:** Se implementó una interfaz intuitiva y *responsive*, capaz de adaptarse a dispositivos móviles. Además, se desarrollaron soluciones dinámicas avanzadas, como un calendario interactivo que bloquea en tiempo real las horas ya reservadas y un sistema de validación visual mediante *tickets* emergentes. *(Anexo 8)*
- **Resolución de problemas técnicos:** Durante el ciclo de desarrollo se superaron retos propios de un entorno real de producción, tales como la prevención del reenvío múltiple de formularios (aplicando el patrón PRG - *Post/Redirect/Get*), la corrección de desbordamientos en CSS (*overflow*) y la gestión correcta del ciclo de vida de las conexiones a la base de datos (patrón *Singleton*).

En definitiva, BarberAPI ha demostrado ser una excelente herramienta para consolidar los conocimientos adquiridos durante el primer curso del ciclo de Desarrollo de Aplicaciones Web (DAW), unificando lenguajes de marcas, programación orientada a objetos y gestión de bases de datos en un único producto de software.

8.2 Mejoras futuras

Dado que el proyecto establece una base arquitectónica sólida, BarberAPI tiene un gran potencial de escalabilidad. De cara a futuras iteraciones o como continuación para el segundo año del ciclo formativo, se proponen las siguientes mejoras:

- **Panel de Administración (Backoffice):** Desarrollo de un portal interno exclusivo para los empleados de la barbería, donde puedan visualizar su agenda diaria, y gestionar (crear, modificar, cancelar) las citas y servicios sin necesidad de interactuar directamente con la base de datos.
- **Sistema de Autenticación y Seguridad:** Implementación de un sistema de registro y login para usuarios (empleando *frameworks* como Spring Security). Esto permitiría a los clientes tener un perfil propio para consultar su historial de cortes y autogestionar la cancelación de sus reservas.
- **Notificaciones Automatizadas:** Integración de una API de terceros para el envío automático de correos electrónicos o mensajes SMS de confirmación y recordatorio 24 horas antes de la cita, reduciendo así la tasa de ausencias.
- **Evolución del Stack Tecnológico:** Migración progresiva de la capa de acceso a datos de JDBC puro a un ORM (como Hibernate/JPA) para optimizar las consultas, y separación del Frontend mediante el uso de *frameworks* modernos como React o Angular, transformando el proyecto en una arquitectura basada en microservicios (API REST pura).

9. Bibliografía y recursos

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Recuperado de https://git-scm.com/book/es/v2

JetBrains. (2024). *IntelliJ IDEA Documentation*. Recuperado de https://www.jetbrains.com/help/idea/

Mozilla Developer Network [MDN]. (2024). *Tecnologías web para desarrolladores: HTML, CSS y JavaScript*. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web

Oracle. (2021). *Java Platform, Standard Edition 17 API Specification*. Oracle Corporation. Recuperado de https://docs.oracle.com/en/java/javase/17/docs/api/

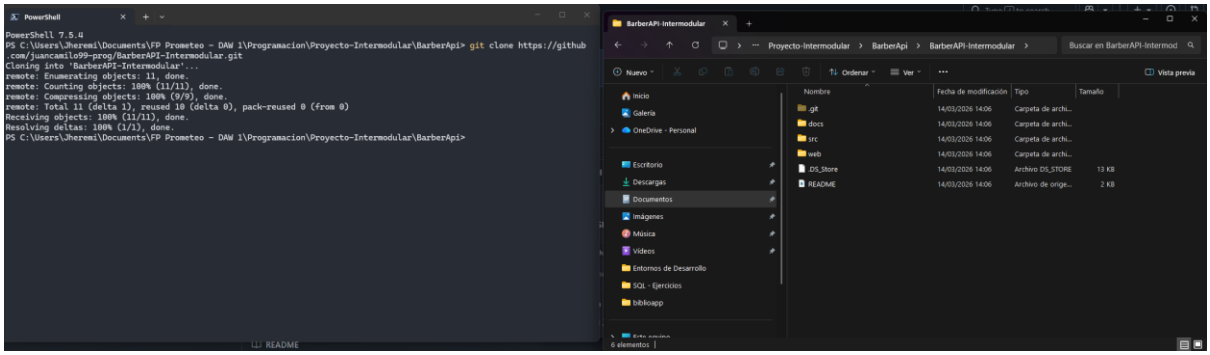
Oracle. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. Recuperado de https://dev.mysql.com/doc/refman/8.0/en/

Spring. (2024). *Spring Boot Reference Documentation*. Spring Framework. Recuperado de https://docs.spring.io/spring-boot/docs/current/reference/html/

The Thymeleaf Team. (2024). *Tutorial: Using Thymeleaf*. Thymeleaf. Recuperado de https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html

10.Anexos

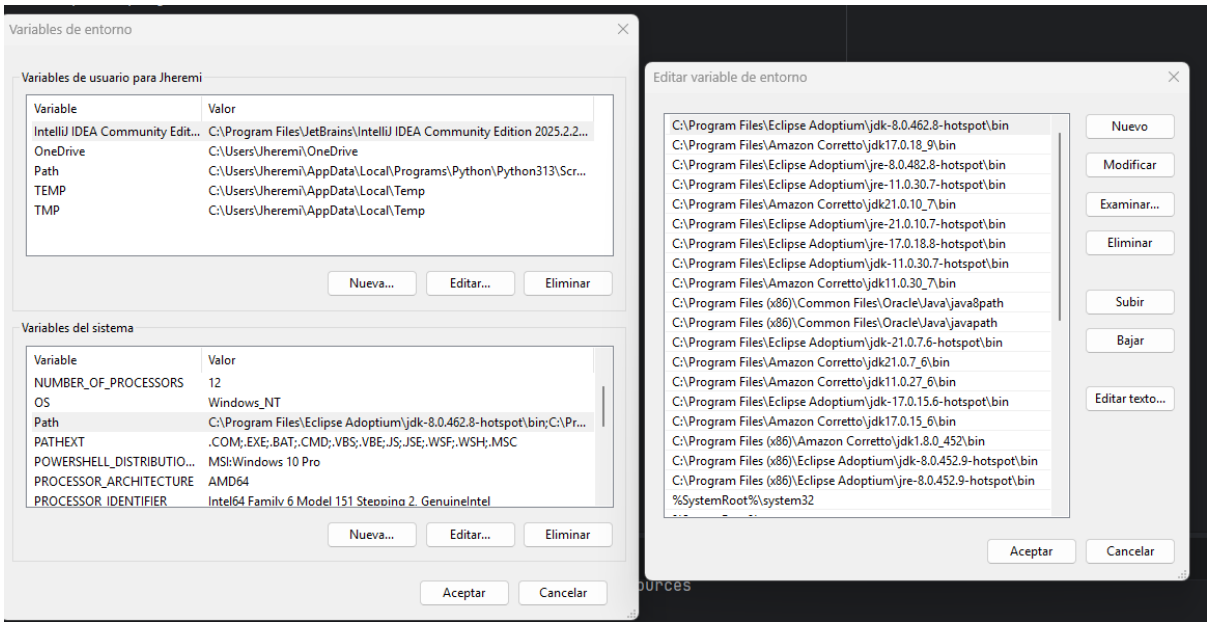
Anexo 1: Clonación *repositorio estructura*.



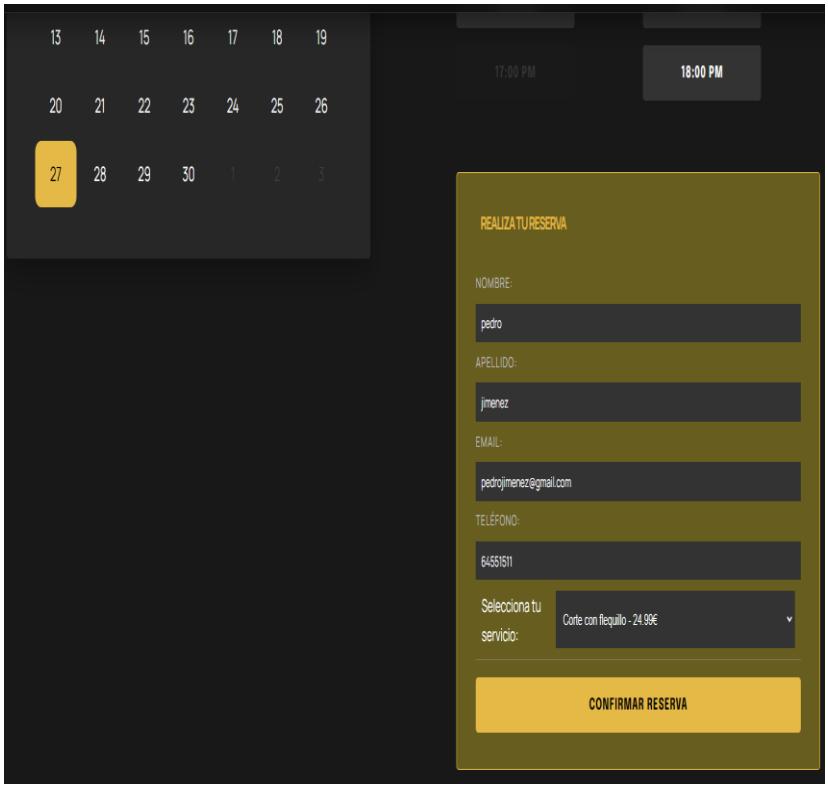
Anexo 2: Error Compilación:

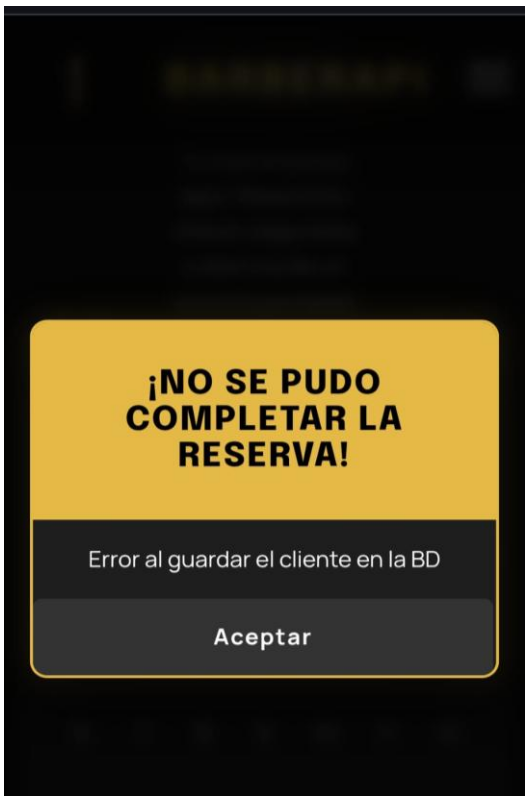
```
Terminal   Local
[INFO] Skip non existing resourceDirectory C:\Users\Jheremi\Documents\FP Prometeo - DAW 1\Programacion\Proyecto-Intermodular\BarberAPI-Intermodular\barberapi\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ barberapi ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 1 source file with javac [debug parameters release 17] to target\test-classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.909 s
[INFO] Finished at: 2026-04-11T21:42:50+02:00
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.14.1:testCompile (default-testCompile) on project barberapi: Fatal error compiling: invalid target release: 17 -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException

PS C:\Users\Jheremi\Documents\FP Prometeo - DAW 1\Programacion\Proyecto-Intermodular\BarberAPI-Intermodular\barberapi>
```

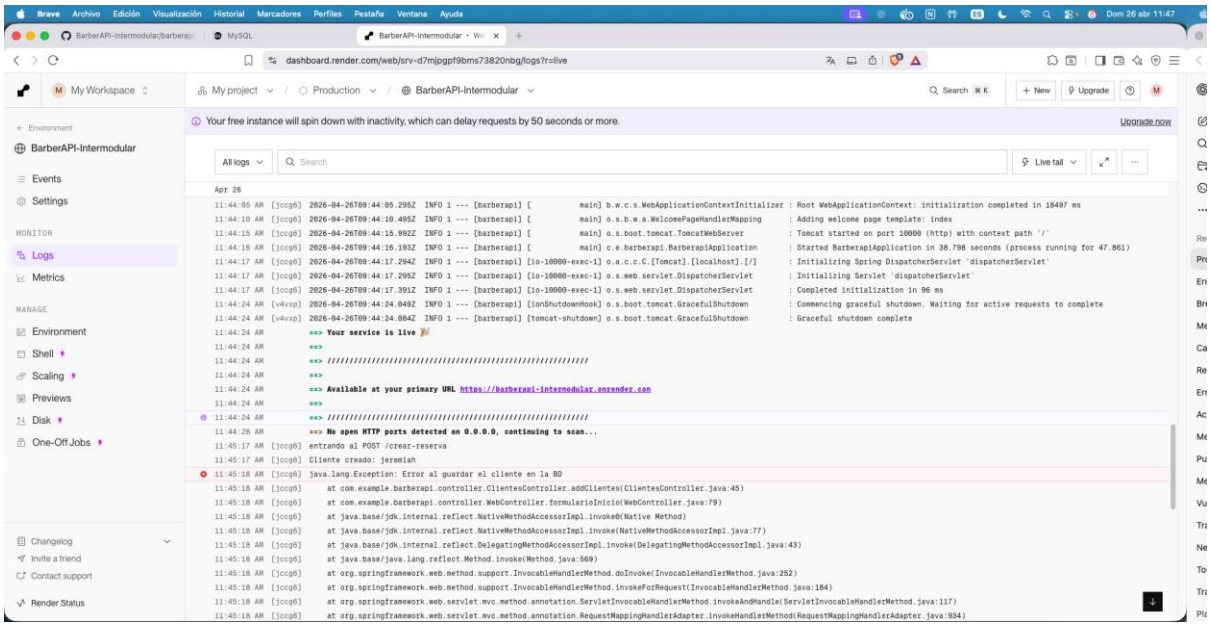
Anexo 3: Variables de entorno.



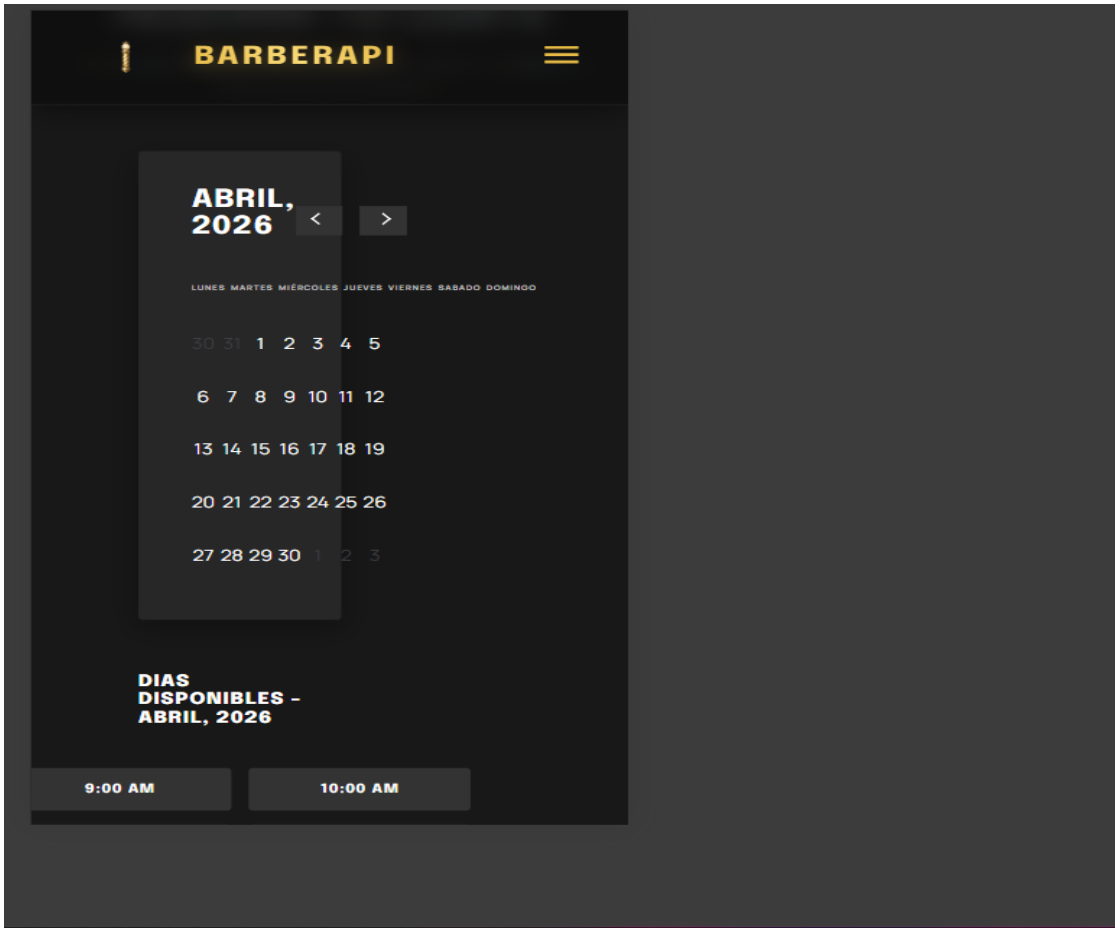
Anexo 4: Prueba reserva – Error DB



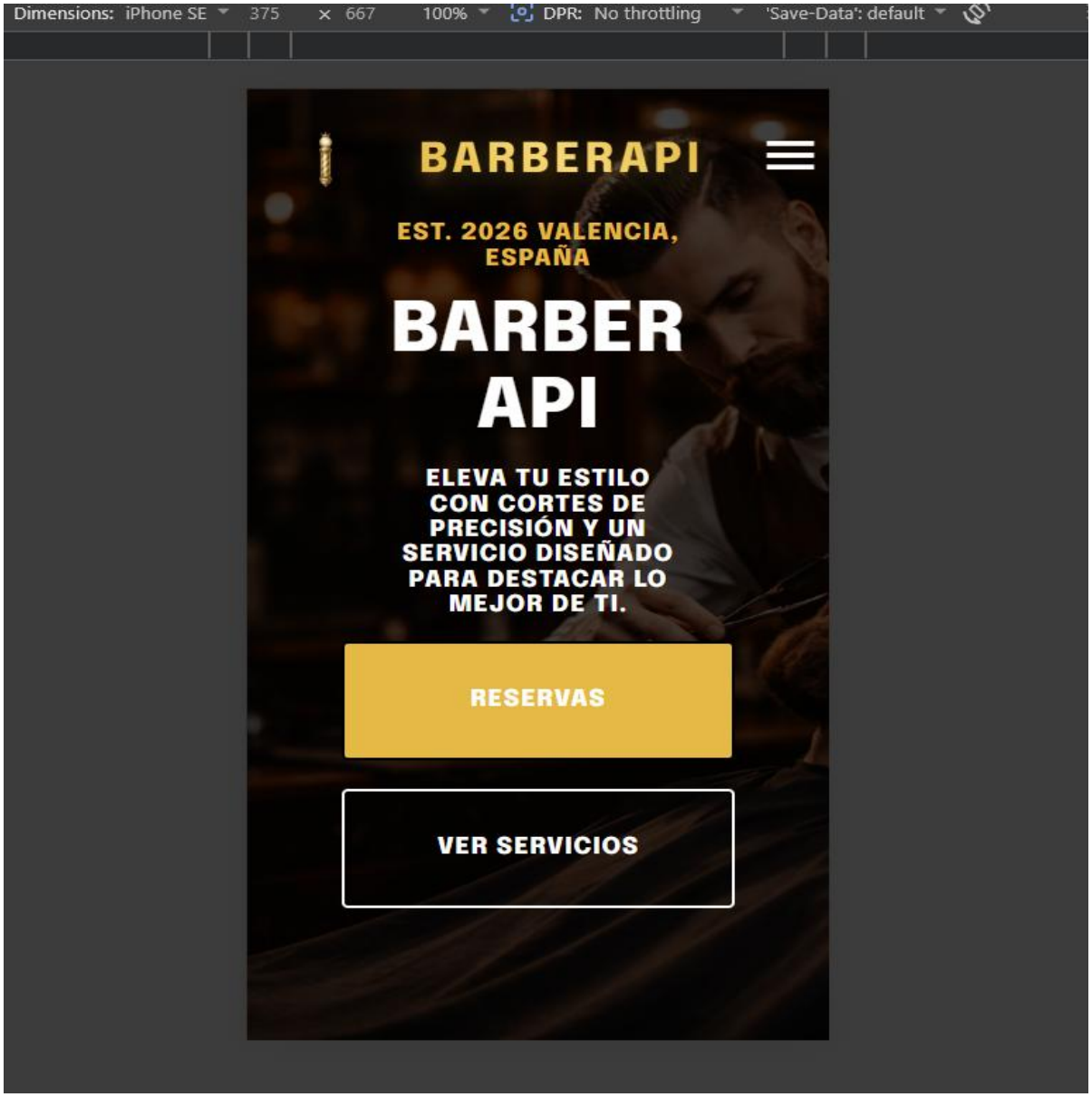


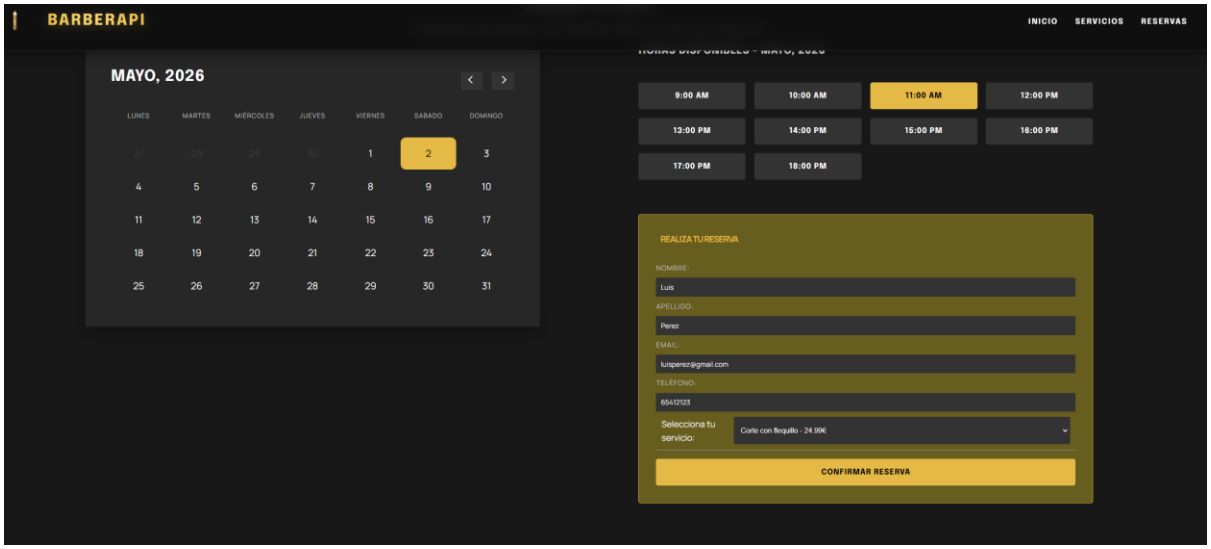


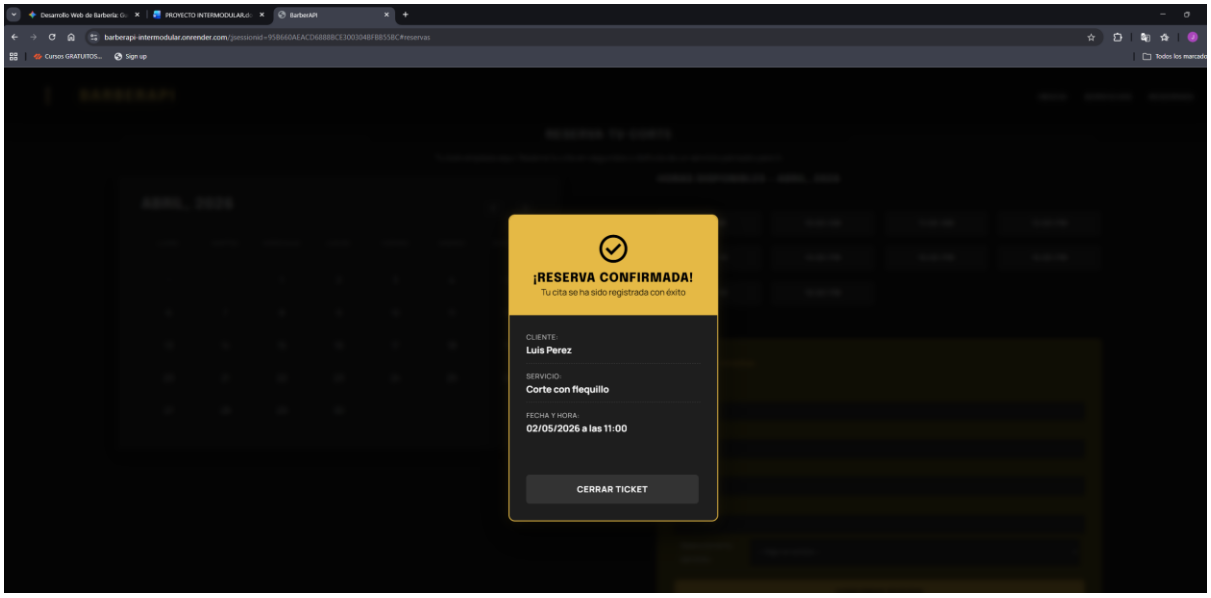
Anexo 5: Desbordamiento maquetación dispositivos pequeños.



Anexo 6: Solución desbordamiento elementos css y html







Anexo 7: Resultados obtenido en la base de datos

Environment

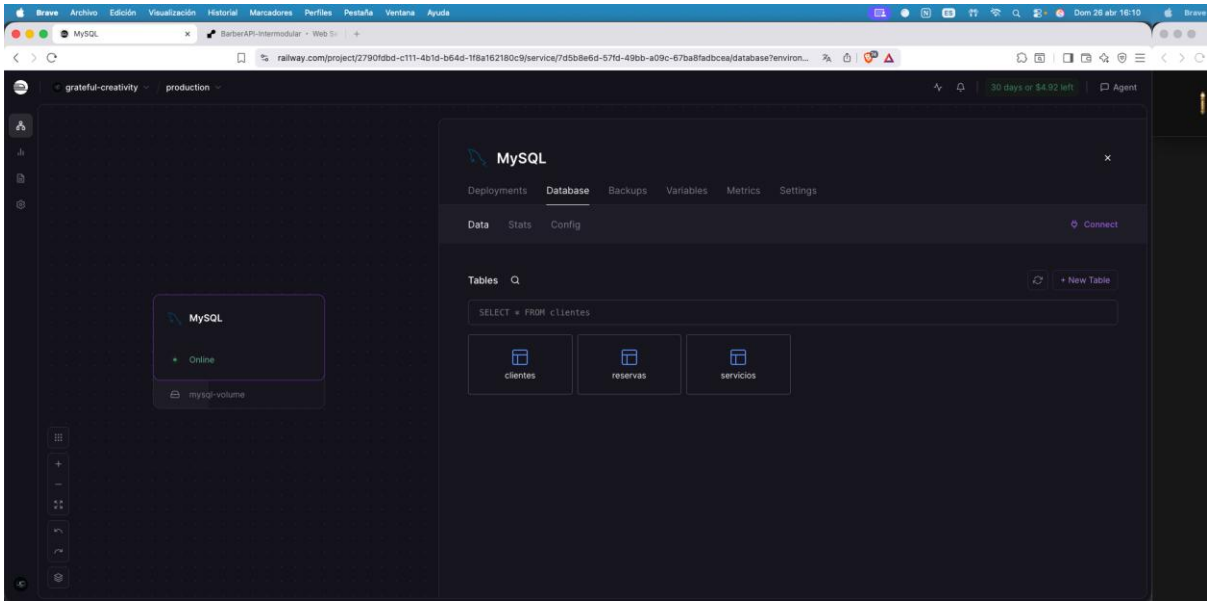
+ Create environment group

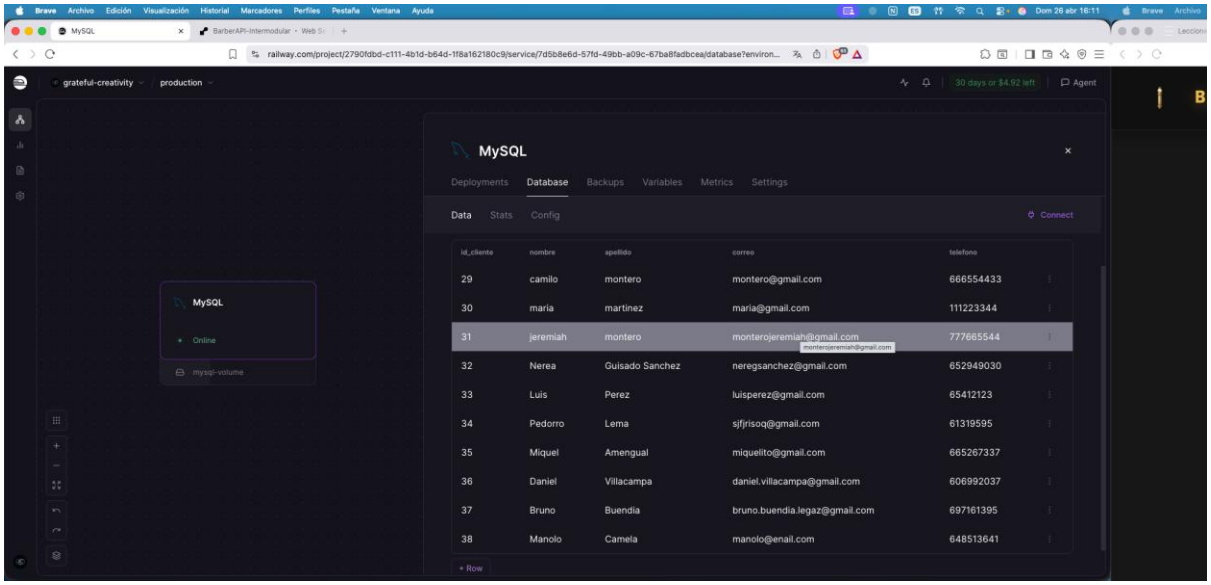
Environment Variables

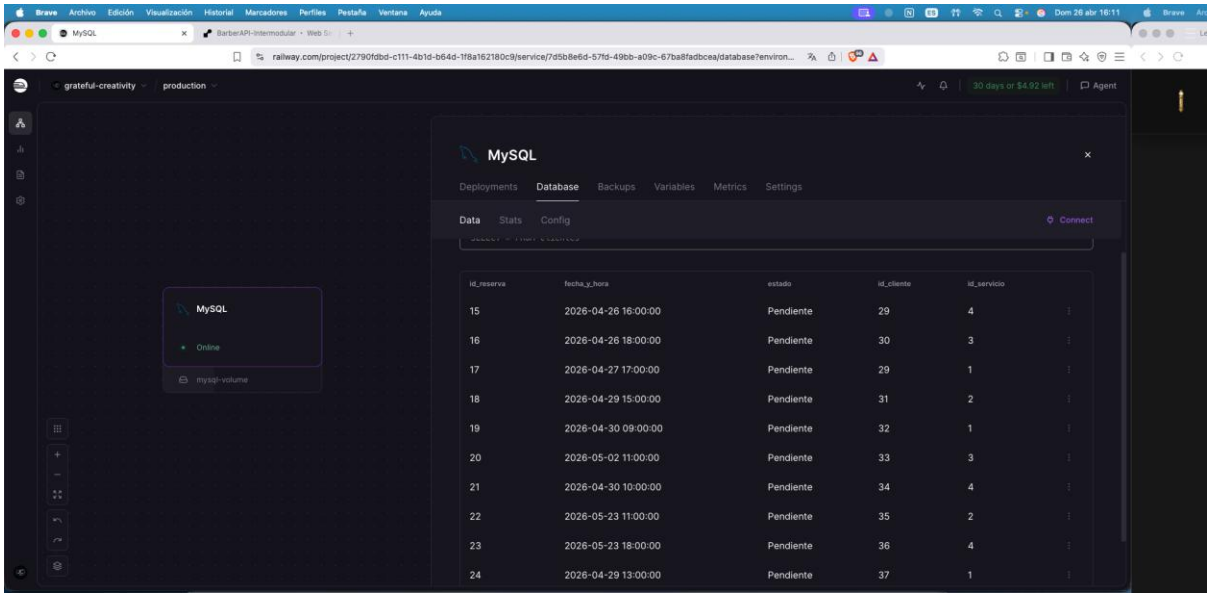
Set environment-specific config and secrets (such as API keys), then read those values from your code. Learn more.

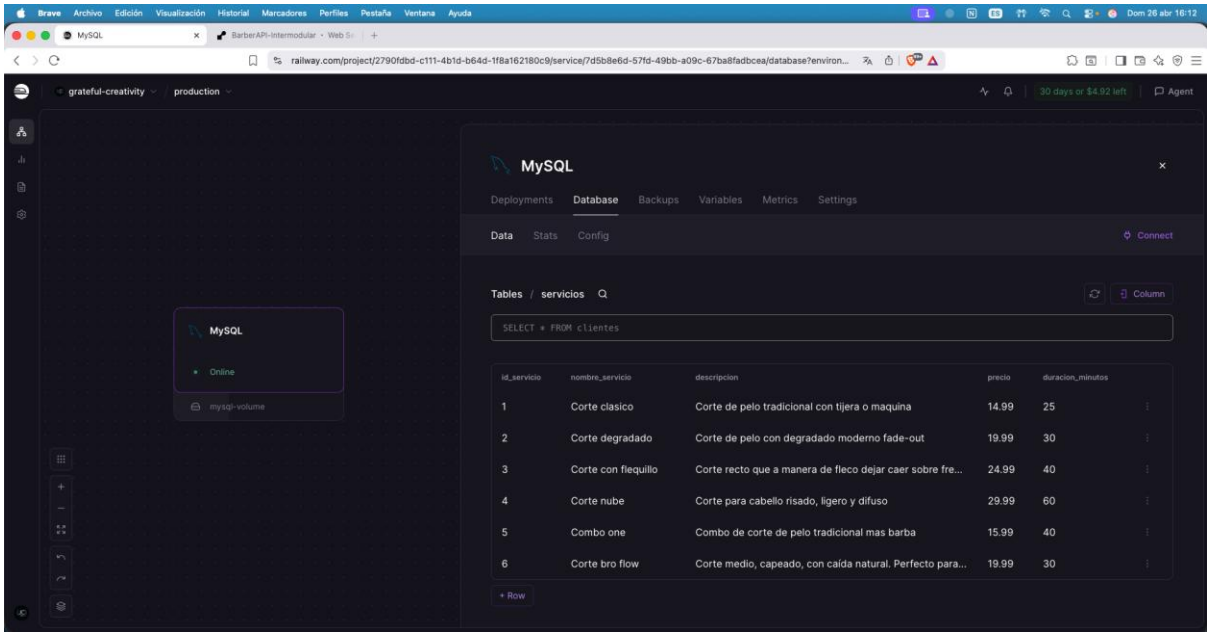
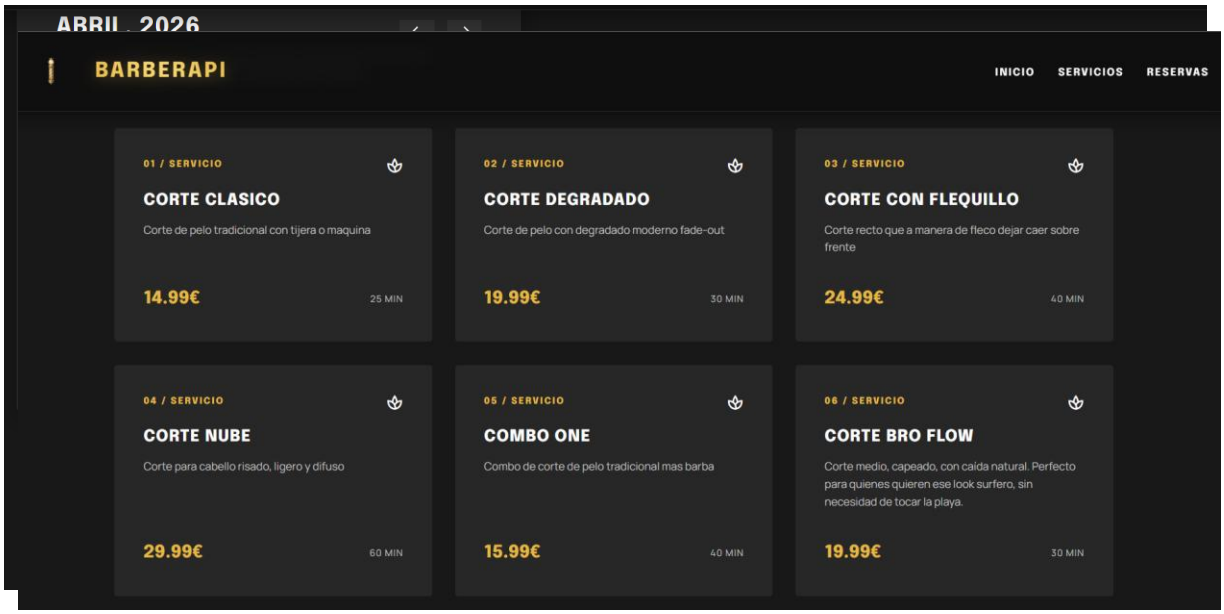
Export | Edit

KEY	VALUE
DB_PASSWORD	SgTOuCObCkIdBaadmvTuJiPBhiWtZemW
DB_URL	jdbc:mysql://shortline.proxy.rlwy.net:38662/railway?allowPublicKeyRetrieval=true&useSSL=false
DB_USER	root


grateful-creativity
production
MySQL
Deployments
Database
Backups
Variables
Metrics
Settings
Data
Stats
Config
Connect
Tables
+ New Table
SELECT * FROM clientes
clientes
reservas
servicios
Online
mysql-volume


grateful-creativity
production
MySQL
Deployments
Database
Backups
Variables
Metrics
Settings
Data
Stats
Config
Connect
id_cliente nombre apellido correo telefono
29 camilo montero montero@gmail.com 666554433
30 maria martinez maria@gmail.com 111223344
31 jeremiah montero monterojeremiah@gmail.com 777665544
32 Nerea Guisado Sanchez neregsanchez@gmail.com 652949030
33 Luis Perez luisperez@gmail.com 65412123
34 Pedorro Lema sjfjrisoq@gmail.com 61319595
35 Miquel Amengual miquelito@gmail.com 665267337
36 Daniel Villacampa daniel.villacampa@gmail.com 606992037
37 Bruno Buendia bruno.buendia.legaz@gmail.com 697161395
38 Manolo Camela manolo@enail.com 648513641
+ Row
Online
mysql-volume


railway.com/project/2790fdbd-c111-4b1d-b64d-1f8a162180c9/service/7d5b8e6d-57fd-49bb-a09c-67ba8fadbcea/database?environ...
grateful-creativity
production
30 days or $4.92 left
Agent
MySQL
Deployments
Database
Backups
Variables
Metrics
Settings
Data
Stats
Config
Connect
MySQL
Online
mysql-volume
id_reserva fecha_y_hora estado id_cliente id_servicio
15 2026-04-26 16:00:00 Pendiente 29 4
16 2026-04-26 18:00:00 Pendiente 30 3
17 2026-04-27 17:00:00 Pendiente 29 1
18 2026-04-29 15:00:00 Pendiente 31 2
19 2026-04-30 09:00:00 Pendiente 32 1
20 2026-05-02 11:00:00 Pendiente 33 3
21 2026-04-30 10:00:00 Pendiente 34 4
22 2026-05-23 11:00:00 Pendiente 35 2
23 2026-05-23 18:00:00 Pendiente 36 4
24 2026-04-29 13:00:00 Pendiente 37 1


railway.com/project/2790fdbd-c111-4b1d-b64d-1f8a162180c9/service/7d5b8e6d-57fd-49bb-a09c-67ba8fadbcea/database?environ...
grateful-creativity
production
30 days or $4.92 left
Agent
MySQL
Deployments
Database
Backups
Variables
Metrics
Settings
Data
Stats
Config
Connect
Tables / servicios
Column
SELECT * FROM clientes
MySQL
Online
mysql-volume
id_servicio nombre_servicio descripcion precio duracion_minutos
1 Corte clasico Corte de pelo tradicional con tijera o maquina 14.99 25
2 Corte degradado Corte de pelo con degradado moderno fade-out 19.99 30
3 Corte con flequillo Corte recto que a manera de fleco dejar caer sobre fre... 24.99 40
4 Corte nube Corte para cabello risado, ligero y difuso 29.99 60
5 Combo one Combo de corte de pelo tradicional mas barba 15.99 40
6 Corte bro flow Corte medio, capeado, con caída natural. Perfecto para... 19.99 30
+ Row

Anexo 8: Comprobación resultado web UX